

DATA CLEANING USING MYSQL WORKBENCH

A Personal Project

by

LOUIE REY GUMERA

September 2026

TABLE OF CONTENTS

PRELIMINARIES	Page
Title Page	i
Table of Contents	ii
CHAPTER 1 – DATA OVERVIEW	1
CHAPTER 2 – DATA CLEANING STEPS	2
1. Creating a backup table	2
1.1. Inserting the data from the initial table to the backup table	2
2. Identifying Duplicates	2
3. Removing white spaces	3
4. Identifying misspellings	3
5. Identifying missing and invalid entries	3
6. Changing incorrect values	4
6.1. Changing incorrect values	5
7. Checking the Price_Per_Unit	6
8. Fixing the date format	7
9. Standardizing datatype	8
CHAPTER 3 – CONCLUSION	9
REFERENCES	10

Chapter 1

DATA OVERVIEW

The dataset that was used in this project is owned by Ahmed Mohamed. The dataset consists of cafe sales including various columns such as Transaction_ID, Item, Quantity, Price_Per_Unit, Total_Spent, Payment_Method, Location, and Transaction_Date. The dataset has a total of 8544 transactions and a total of 8545 rows. The dataset that was used in this project contained inconsistencies such as white spaces, incorrect data formats, invalid values and missing values that were needed to be addressed to ensure data integrity for further analysis.

Chapter 2

DATA CLEANING STEPS

1. Creating a backup table

A backup table is necessary before doing any changes, it keeps the original table intact for future revisions.

```
CREATE TABLE `sales_staging` (
  `Transaction_ID` text,
  `Item` text,
  `Quantity` int DEFAULT NULL,
  `Price_Per_Unit` double DEFAULT NULL,
  `Total_Spent` double DEFAULT NULL,
  `Payment_Method` text,
  `Location` text,
  `Transaction_Date` text
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

1.1. Inserting the data from the initial table to the backup table

```
INSERT INTO sales_staging
SELECT *
FROM cafe_sales;
```

2. Identifying duplicates

Identifying duplicates helps keep the dataset clean and reliable. In this dataset, the author did not find any duplicates.

```
WITH duplicate_test AS(
  SELECT *,
  ROW_NUMBER() OVER(PARTITION BY Transaction_ID) AS Row_Num
  FROM sales_staging
)
SELECT COUNT(Row_Num)
FROM duplicate_test
WHERE Row_Num > 1;
```

3. Removing white spaces

Removing white spaces in a dataset ensures that text values are clean and consistent.

```
UPDATE sales_staging  
SET Transaction_ID = TRIM(Transaction_ID);
```

```
UPDATE sales_staging  
SET Item = TRIM(Item);
```

```
UPDATE sales_staging  
SET Payment_Method = TRIM(Payment_Method);
```

```
UPDATE sales_staging  
SET Location = TRIM(Location);
```

4. Identifying misspellings

Identifying misspellings finds and correct incorrectly spelled values so that the data remains accurate and consistent. The author did not find any misspellings but found invalid entries including “ERROR” and “UNKNOWN”.

```
SELECT DISTINCT Item  
FROM sales_staging;
```

```
SELECT DISTINCT Payment_Method  
FROM sales_staging;
```

```
SELECT DISTINCT Location  
FROM sales_staging;
```

5. Identifying missing and invalid entries

The author identified missing and invalid entries finds incomplete or incorrect values that could affect the accuracy of the data and future analysis. The author found a total of 7,261 missing and invalid entries including “ERROR” and “UNKNOWN”. 812 in

Item column, 2,691 in Payment_Method column, 3,373 in Location column, and 385 in Transaction_Date column.

```
SELECT
    SUM(Transaction_ID IS NULL OR Transaction_ID = '' OR Transaction_ID =
'UNKNOWN' OR Transaction_ID = 'ERROR') AS Transaction_ID_Missing,
    SUM(Item IS NULL OR Item = '' OR Item = 'UNKNOWN' OR Item = 'ERROR')
AS Item_Missing,
    SUM(Quantity IS NULL OR Quantity = '' OR Quantity = 'UNKNOWN' OR
Quantity = 'ERROR') AS Quantity_Missing,
    SUM(Price_Per_Unit IS NULL OR Price_Per_Unit = '' OR Price_Per_Unit =
'UNKNOWN' OR Price_Per_Unit = 'ERROR') AS Price_Per_Unit_Missing,
    SUM(Total_Spent IS NULL OR Total_Spent = '' OR Total_Spent = 'UNKNOWN'
OR Total_Spent = 'ERROR') AS Total_Spent_Missing,
    SUM(Payment_Method IS NULL OR Payment_Method = '' OR Payment_Method =
'UNKNOWN' OR Payment_Method = 'ERROR') AS Payment_Method_Missing,
    SUM(Location IS NULL OR Location = '' OR Location = 'UNKNOWN' OR
Location = 'ERROR') AS Location_Missing,
    SUM(Transaction_Date IS NULL OR Transaction_Date = '' OR
Transaction_Date = 'UNKNOWN' OR Transaction_Date = 'ERROR') AS
Transaction_Date_Missing
FROM sales_staging;
```

6. Changing incorrect values

The author changed incorrect values in Item column including the blank values and null. Given that the price for each item is given, the author makes use of it to fix the incorrect and blank values including null using Case statement.

```
UPDATE sales_staging
SET
    Item = CASE
        WHEN Price_Per_Unit = 2 THEN 'Coffee'
        WHEN Price_Per_Unit = 1.5 THEN 'Tea'
        WHEN Price_Per_Unit = 5 THEN 'Salad'
        WHEN Price_Per_Unit = 1 THEN 'Cookie'
    ELSE Item
END;
```

6.1. Changing incorrect values

The author changed incorrect values in all columns that has missing and invalid entries including Item column, Payment_Method column, and Location column to maintain consistency in data. The author changed “ERROR”, “UNKNOWN”, null, and blank values into “NaN”. But the author changed “ERROR”, “UNKNOWN”, and blank values into NULL in the Transaction_Date column to ensure that the author will not face any errors in the future with the date format.

```
UPDATE sales_staging
```

```
SET
```

```
    Item = CASE
```

```
        WHEN Item = 'UNKNOWN' THEN 'NaN'
```

```
        WHEN Item = 'ERROR' THEN 'NaN'
```

```
        WHEN Item = '' THEN 'NaN'
```

```
        WHEN Item IS NULL THEN 'NaN'
```

```
    ELSE Item
```

```
END;
```

```
UPDATE sales_staging
```

```
SET
```

```
    Payment_Method = CASE
```

```
        WHEN Payment_Method = 'UNKNOWN' THEN 'NaN'
```

```
        WHEN Payment_Method = 'ERROR' THEN 'NaN'
```

```
        WHEN Payment_Method = '' THEN 'NaN'
```

```
        WHEN Payment_Method IS NULL THEN 'NaN'
```

```
    ELSE Payment_Method
```

```
END;
```

```
UPDATE sales_staging
```

```
SET
```

```
    Location = CASE
```

```
        WHEN Location = 'UNKNOWN' THEN 'NaN'
```

```
        WHEN Location = 'ERROR' THEN 'NaN'
```

```
        WHEN Location = '' THEN 'NaN'
```

```
        WHEN Location IS NULL THEN 'NaN'
```

```
    ELSE Location
```

```
END;
```

```

UPDATE sales_staging
SET
    Transaction_Date = CASE
        WHEN Transaction_Date = 'UNKNOWN' THEN NULL
        WHEN Transaction_Date= 'ERROR' THEN NULL
        WHEN Transaction_Date = '' THEN NULL
    ELSE Transaction_Date
    END;

```

7. Checking the Price_Per_Unit column

The author wanted to make sure that the Price_Per_Unit column does not have inconsistencies. The author used Case statements to ensure that the data is accurate. The author did not find any incorrect price.

```

SELECT
    Transaction_ID, Item, Price_Per_Unit AS Price_On_Transaction,
    CASE
        WHEN Item = 'Coffee' THEN 2
        WHEN Item = 'Tea' THEN 1.5
        WHEN Item = 'Sandwich' THEN 4
        WHEN Item = 'Salad' THEN 5
        WHEN Item = 'Cake' THEN 3
        WHEN Item = 'Cookie' THEN 1
        WHEN Item = 'Smoothie' THEN 4
        WHEN Item = 'Juice' THEN 3
    END AS Expected_Price,
    CASE
        WHEN Price_Per_Unit =
            CASE
                WHEN Item = 'Coffee' THEN 2
                WHEN Item = 'Tea' THEN 1.5
                WHEN Item = 'Sandwich' THEN 4
                WHEN Item = 'Salad' THEN 5
                WHEN Item = 'Cake' THEN 3
                WHEN Item = 'Cookie' THEN 1
                WHEN Item = 'Smoothie' THEN 4
                WHEN Item = 'Juice' THEN 3
            END
        THEN 'CORRECT'
        ELSE 'INCORRECT'
    END AS Price_Check
FROM sales_staging;

```

```

WITH Price_Checker AS (
SELECT
Transaction_ID, Item, Price_Per_Unit AS Price_On_Transaction,
CASE
    WHEN Item = 'Coffee' THEN 2
    WHEN Item = 'Tea' THEN 1.5
    WHEN Item = 'Sandwich' THEN 4
    WHEN Item = 'Salad' THEN 5
    WHEN Item = 'Cake' THEN 3
    WHEN Item = 'Cookie' THEN 1
    WHEN Item = 'Smoothie' THEN 4
    WHEN Item = 'Juice' THEN 3
END AS Expected_Price,
CASE
    WHEN Price_Per_Unit =
        CASE
            WHEN Item = 'Coffee' THEN 2
            WHEN Item = 'Tea' THEN 1.5
            WHEN Item = 'Sandwich' THEN 4
            WHEN Item = 'Salad' THEN 5
            WHEN Item = 'Cake' THEN 3
            WHEN Item = 'Cookie' THEN 1
            WHEN Item = 'Smoothie' THEN 4
            WHEN Item = 'Juice' THEN 3
        END
    THEN 'CORRECT'
    ELSE 'INCORRECT'
END AS Price_Check
FROM sales_staging)
SELECT COUNT(Price_Check) AS INCORRECT_COUNT
FROM Price_Checker
WHERE Price_Check = 'INCORRECT'
AND Item != 'NaN';

```

8. Fixing the date format

The author fixed the date format to ensure that all date values follow a consistent and valid format.

```

UPDATE sales_staging
SET Transaction_Date = STR_TO_DATE(Transaction_Date, '%Y-%m-%d');

```

9. Standardizing datatype

The author changed the datatype of dates to ensure that the Transaction_Date column stores values using the appropriate and consistent datatype.

```
ALTER TABLE sales_staging  
MODIFY COLUMN Transaction_Date DATE;
```

Chapter 3

CONCLUSION

The dataset was cleaned to improve its overall quality, accuracy, and consistency. The cleaning process included checking for duplicates, identifying missing and invalid entries, correcting misspellings, removing unnecessary white spaces, fixing date formats, and standardizing data types. These steps helped ensure that the data was properly structured and reliable for further analysis. After cleaning, the dataset is more consistent and suitable for performing accurate SQL queries and generating meaningful insights.

REFERENCES

Ahmed Mohamed (2024). *Cafe Sales - Dirty Data for Cleaning Training* from <https://www.kaggle.com/datasets/ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training>